VulnSight — System Design & Project Proposal

Project title: VulnSight (Web-Vuln-Scanner)

Type: Modular authorized web vulnerability scanner (CLI)

Group: Group 3

Language: Python 3.10+

License: MIT

Student name	Student ID
Kenean Amare Eshet	CTC-4837-26
Kenneti Hailu Dange	CTC-535-26
Kidsit Ephrem	CTC-6744-26
Alazar Elias Digafe	CTC-631-26
Firaol Mustefa Gossa	CTC-4301-26

---

1. Abstract

VulnSight is a modular command-line web vulnerability scanner designed for **authorized local assessments**. It integrates reconnaissance, multiple detection engines, and professional reporting into one controlled pipeline. The system is built as a five-module monorepo so each team member can own a clear functional area while sharing a common findings contract.

Operators authorize a target with a project file, crawl the same-origin application, run detection modules (XSS, SQL injection, command injection, access control), then generate HTML, Markdown, PDF, and JSON reports with severity and verification status.

---

2. Problem statement

Web applications commonly expose injection and access-control flaws. Academic and training environments need a scanner that:

- Is **ethically constrained** (authorization-first, lab/loopback by default).
- Is **modular** so a team can develop and test components independently.
- Produces **honest, reviewable reports** (evidence, confidence, limitations) rather than overclaiming exploits.
- Remains usable as a **CLI tool** suitable for demonstration and submission.

---

3. Objectives

Objective	How VulnSight addresses it
Explicit authorization	`init-project` / `scan --project` records permission and target
End-to-end pipeline	Orchestrator runs recon → detect → report
Multi-class detection	XSS, SQLi, command injection, access control
Clear modular ownership	Separate directories with public APIs
Useful outputs	HTML (interactive), Markdown, PDF, JSON
Safe defaults	Loopback-only unless operator opts into external scope

---

4. Scope

In scope

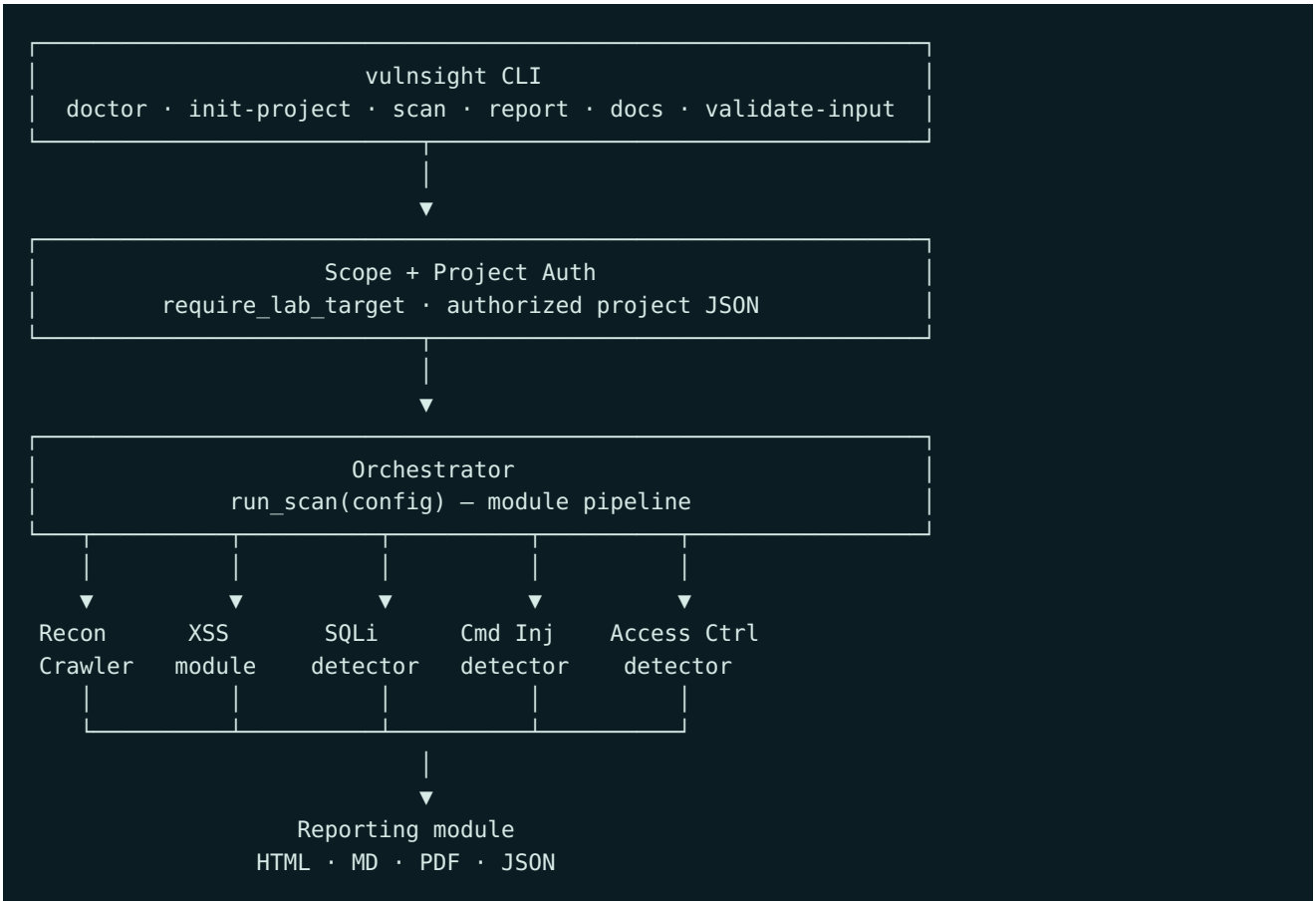
- Authorized HTTP targets (default: `127.0.0.1` / `localhost` / `::1`)
- Same-origin crawling and form/parameter discovery
- Detection of selected OWASP-related classes (XSS, injection, broken access control / IDOR)
- Normalization, conservative deduplication, CVSS scoring where possible
- CLI orchestration and documentation

Out of scope

- Shipping a bundled deliberately vulnerable target application
- Unauthorized scanning of third-party or production systems
- Guaranteeing complete coverage of all vulnerability classes
- Claiming confirmed exploitation without supporting evidence

---

5. System overview



High-level stages

- **Authorize** — create/load project file with target and permission attestation.
- **Recon** — crawl same-origin pages/forms into `crawl_map.json`.
- **Detect** — run detection modules against the live target (and crawl map where applicable).
- **Report** — normalize findings, score, dedupe, write professional reports.

---

6. Repository structure

Path	Responsibility
`vulnsight/`	CLI, project authorization, scope checks, orchestrator, terminal UI
`Crawler/`	Same-origin reconnaissance → `crawl_map.json`
`xss_module/`	Reflected / stored XSS analysis
`sqli_&_command_module/`	SQL injection and OS command-injection checks
`access_control_module/`	IDOR and broken access control checks
`reporting_module/`	Ingestion, CVSS, HTML / Markdown / PDF / JSON reports
`docs/`	Usage, architecture, demo, and this design proposal
`tests/`	Integration and CLI/unit tests

---

7. How each functionality works

7.1 CLI and orchestration (`vulnsight/`)

Purpose: Provide a single entry point and run the full pipeline safely.

Component	Role
`cli.py`	Commands: `doctor`, `init-project`, `scan`, `report`, `docs`, `validate-input`, `version`
`project.py`	Loads/writes authorized project JSON (target, modules, budgets, authorization fields)
`scope.py`	Enforces lab-first URL rules; optional `--allow-external` for explicitly authorized non-loopback hosts
`orchestrator.py`	Calls recon → detection modules → reporting; preserves findings if some modules fail
`ui.py`	Terminal banner, module status lines, scan summary
`config.py` / `paths.py`	Defaults and module import paths

Typical operator flow

```
vulnsight init-project -o myproject.json --target http://127.0.0.1:5000
vulnsight scan --project myproject.json
```

7.2 Authorization project file

Purpose: Make permission explicit before scanning.

The project JSON stores:

- Target URL and authorized origins
- Authorization attestation and `authorized_by`
- Optional modules, budgets (`max_pages`, `timeout`, `max_requests`)
- Output directory and report formats

Crawler-discovered links are **not** treated as authorization. Only the configured target/project scope is.

7.3 Reconnaissance — Crawler (`Crawler/`)

Purpose: Map the application surface for later detectors (especially XSS).

How it works

- Starts from the seed URL (target).
- Fetches HTML pages within the **same origin**.
- Extracts links, forms, and query parameters.
- Writes `crawl_map.json` (schema v1.0) containing `pages`, `forms`, and `endpoints`.

Public interface: `run_crawler_module(...)`

Output: `reports/crawl_map.json`

Safety notes: loopback/private checks, redirect re-validation, page/request budgets.

7.4 XSS module (`xss_module/`)

Purpose: Detect reflected and stored cross-site scripting candidates.

How it works

- Loads `crawl_map.json` and extracts injectable input points (query params, form fields).
- Sends marker / probe values through those inputs.
- Analyzes responses for reflection context (HTML body, attributes, etc.).
- Optionally exercises stored sinks (e.g. comment-style forms).
- Emits `Finding` objects with confidence and verification status (`candidate` / `probable` / etc.).

Public interface: `run_xss_module(...)`

Output: `reports/findings_xss.json`

Design principle: confirmation is **earned** from evidence; the module avoids inventing "confirmed XSS" without support.

7.5 SQL injection (`sqli_&_command_module/sqli/`)

Purpose: Identify possible SQL injection, especially authentication bypass and error-based signals.

How it works

- Probes login-style endpoints (legacy `/login` and crawl-discovered login forms).
- Compares baseline invalid login vs controlled SQLi payloads (behavioral bypass).
- Uses crawl-map forms/params for **error-based** probes (database error signatures in responses).
- Records findings with evidence (status codes, redirects, matched signatures).

Public interface: `check_local_target(base_url, crawl_map=...)`

Output: `reports/findings_sqli.json`

7.6 Command injection (`sqli_&_command_module/command_injection/`)

Purpose: Detect unsafe command-concatenation sinks.

How it works

- Probes classic `/ping`-style sinks and crawl-discovered host/cmd-like parameters.
- Injects a unique marker through the parameter.
- If the marker appears in the response (and not in the baseline), records a finding.
- Labels lab/simulation evidence carefully — marker echo indicates an unsafe pattern, not necessarily live OS shell execution.

Public interface: `check_local_target(base_url, crawl_map=...)`

Output: `reports/findings_command_injection.json`

7.7 Access control (`access_control_module/`)

Purpose: Detect IDOR and broken access control on privileged routes.

How it works

- Discovers profile/admin-style paths from defaults and the crawl map.
- Attempts authenticated cross-user profile access (IDOR) when lab credentials work.
- Checks whether privileged paths (e.g. `/admin`) are reachable without proper role checks.
- Also runs **unauthenticated** fallbacks (readable profiles / admin without login) for bring-your-own labs.

Public interface: `check_local_target(base_url, crawl_map=...)`

Output: `reports/findings_access_control.json`

7.8 Reporting (`reporting_module/`)

Purpose: Turn module JSON into a single professional assessment package.

Pipeline

```
Ingest findings → Normalize to canonical schema → Deduplicate
→ Apply CVSS (when possible) → Summarize → Write artifacts
```

Output	Description
`vulnsight_report.html`	Interactive report (filters, search, download buttons)
`vulnsight_report.md`	Markdown report for documents / submission packs
`vulnsight_report.pdf`	PDF report (WeasyPrint or ReportLab)
`findings.json` / `normalized_findings.json`	Machine-readable findings

Each finding includes title, severity, verification status, URL/parameter, CWE/OWASP when available, description, impact, remediation, and evidence.

Public interface: `run_report_module(...)`

---

8. Shared data contracts

Detection modules emit a common shape via `scanner.models`:

- **`Finding`** — vulnerability_type, title, url, method, parameter, evidence, confidence, severity, verification_status, remediation, CWE, module, …
- **`ScanResult`** — `findings[]`, `outcomes[]`, `errors[]`, `stats`, `limitations`

Reporting converts these into a **canonical finding schema**, then scores and deduplicates conservatively so the final report does not invent severity or confirmation.

---

9. End-to-end data flow

```
myproject.json ──► vulnsight scan
                      │
                      ├─► crawl_map.json
                      ├─► findings_xss.json
                      ├─► findings_sqli.json
                      ├─► findings_command_injection.json
                      ├─► findings_access_control.json
                      ├─► findings_raw.json  (combined pre-report)
                      └─► reporting_module
                              ├─► vulnsight_report.html
                              ├─► vulnsight_report.md
                              ├─► vulnsight_report.pdf
                              └─► findings.json
```

If one detection module fails, the orchestrator keeps findings from successful modules and marks coverage as **partial**.

---

10. Safety and ethics design

Control	Behavior
Explicit target	Scan requires `--target` or `--project`

Lab-first scope	Default allows only loopback HTTP hosts
Authorization record	Project file stores permission attestation
Discovery ≠ permission	Crawl map URLs do not expand legal scope
Honest reporting	Limitations and verification status are preserved
No bundled attack lab	Operators bring their own authorized target

---

11. Technology stack

Layer	Technology
Language	Python 3.10+
HTTP client	`requests`
HTML parsing	BeautifulSoup (optional lxml)
Reporting templates	Jinja2
PDF	WeasyPrint and/or ReportLab
Packaging / CLI	`pyproject.toml`, console script `vulnsight`
Tests	`pytest`

---

12. Testing strategy

- **Unit tests** per module (detectors, crawler schema, reporting normalization).
- **CLI smoke tests** for doctor/docs/validate/report commands.
- **Optional live tests** when `VULNSIGHT_LAB_URL` points at an authorized local site.

```
python -m pytest
```

---

13. Demonstration scenario

- Start an authorized local vulnerable website (example: `http://127.0.0.1:5000`).
- Create `myproject.json` with `vulnsight init-project`.
- Run `vulnsight scan --project myproject.json`.
- Open `reports/vulnsight_report.html` (download Markdown/PDF as needed).
- Walk through findings by module (XSS, SQLi, command injection, access control) and discuss remediation.

Reference screenshots: `docs/images/demo-scan-terminals.png`, `demo-report-summary.png`, `demo-report-findings.png`.

---

14. Limitations

- Coverage depends on crawl reachability and target behavior.
- Some findings remain candidates/probable until stronger verification.
- CVSS may be unscored when a complete vector cannot be justified.
- Detectors are educational/bounded; they are not a substitute for a full professional pentest.
- Absence of findings is **not** proof that an application is secure.

---

15. Conclusion

VulnSight delivers a complete, ethically constrained web vulnerability assessment pipeline suitable for a modular graduation project. The design separates concerns across reconnaissance, detection, and reporting, while the orchestrator and authorization model keep scanning intentional, reviewable, and safe for academic demonstration.

---

16. References (project documents)

Document	Path

Usage	`docs/USAGE.md`
Installation	`docs/INSTALLATION.md`
Architecture	`docs/ARCHITECTURE.md`
Demo runbook	`docs/DEMO_RUNBOOK.md`
Security policy	`docs/SECURITY.md` / `SECURITY.md`
Project summary	`docs/PROJECT.md`
Authors	`AUTHORS.md`